Save as:

`Java_Level_6_Multithreading_Concurrency.md`

📘 Java Level 6 — Multithreading & Concurrency

Overview

Level 6 focuses on **running Java code concurrently and asynchronously**.

The most important goal is to understand how to convert code from:

```
Synchronous
    ↓
Task 1
    ↓
Task 2
    ↓
Task 3
    ↓
Response
```

to:

```
Asynchronous / Concurrent
          ↓
    ┌─────┼─────┐
    ↓     ↓     ↓
 Task1  Task2  Task3
    └─────┼─────┘
          ↓
       Response
```

This is especially important for backend applications where multiple external services, database calls, APIs, or I/O operations can execute concurrently.

---

1. Process vs Thread

A **process** is an executing application.

A **thread** is an execution path inside a process.

Example:

```
Java Application
      |
      ├── Thread 1
      ├── Thread 2
      ├── Thread 3
      └── Thread 4
```

Multiple threads can execute tasks concurrently.

---

2. Creating a Thread

The simplest way:

```
Thread thread = new Thread(() -> {

    System.out.println(
            "Running on: "
                    + Thread.currentThread().getName()
    );

});

thread.start();
```

Important:

```
thread.start();
```

starts a **new thread**.

Don't confuse it with:

```
thread.run();
```

Calling `run()` directly does **not** start a new thread. It executes the method on the current thread.

---

3. Runnable

3. Runnable

`Runnable` represents a task that doesn't return a result.

```
Runnable task = () -> {

    System.out.println(
            "Task running on: "
                    + Thread.currentThread().getName()
    );

};

Thread thread = new Thread(task);

thread.start();
```

Mental model:

```
Runnable
   ↓
Task
   ↓
Thread
   ↓
Execution
```

---

4. `Thread.sleep()`

Used to pause the current thread.

```
Thread.sleep(1000);
```

means:

```
Pause current thread
for approximately 1000 ms
```

It throws:

```
InterruptedException
```

Correct handling:

```
try {

    Thread.sleep(1000);

} catch (InterruptedException e) {

    Thread.currentThread().interrupt();
}
```

Important

Don't silently swallow interruption:

```
catch (InterruptedException e) {
    // ❌
}
```

Prefer:

```
Thread.currentThread().interrupt();
```

---

5. Why Manually Creating Threads Doesn't Scale

You could do:

```
new Thread(task1).start();
new Thread(task2).start();
new Thread(task3).start();
```

But a production application may have:

```
1000 requests
      ↓
1000 threads
```

Creating and destroying threads has overhead.

Instead, use a **thread pool**.

6. ExecutorService

`ExecutorService` manages threads for us.

```
ExecutorService executor =
        Executors.newFixedThreadPool(4);
```

This creates a pool containing up to four worker threads.

Submit tasks:

```
executor.submit(() -> {

    System.out.println(
            "Running on: "
                    + Thread.currentThread().getName()
    );

});
```

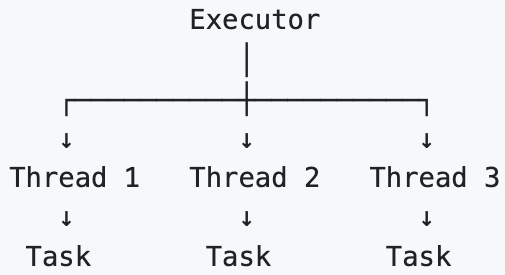
Shutdown:

```
executor.shutdown();
```

7. Fixed Thread Pool

```
ExecutorService executor =
        Executors.newFixedThreadPool(4);
```

Mental model:



If there are more tasks than threads:

```
4 threads
10 tasks
```

the remaining tasks wait until a thread becomes available.

---

8. Callable

`Runnable` doesn't return a result.

`Callable` can return a result.

```
Callable<String> task = () -> {

    return "Hello";
};
```

Submit it:

```
Future<String> future =
        executor.submit(task);
```

Get the result:

```
String result = future.get();
```

Important:

```
future.get()
```

is blocking.

---

9. Future

A `Future` represents the result of an asynchronous computation.

```
Future<String> future =
```

```
        executor.submit(() -> {

            Thread.sleep(1000);

            return "User data";
        });
```

Later:

```
String result = future.get();
```

Conceptually:

```
Task
 ↓
Future
 ↓
Result available later
```

Problem with Future

It's difficult to compose multiple futures.

For example:

```
Future A
   ↓
Future B
   ↓
Future C
```

This is where `CompletableFuture` becomes much more useful.

---

10. CompletableFuture

`CompletableFuture` represents an asynchronous computation that can be **combined, transformed, and chained**.

Basic example:

```
CompletableFuture<String> future =
        CompletableFuture.supplyAsync(
                () -> "Hello"
```

```
        );
```

Get result:

```
String result = future.join();
```

---

11. `supplyAsync()`

Use `supplyAsync()` when the async operation **returns a value**.

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> userService.getUser(userId)
        );
```

Mental model:

```
supplyAsync()
     ↓
Run task asynchronously
     ↓
Return CompletableFuture<T>
```

For example:

```
CompletableFuture<String>
```

means the future will eventually produce a `String`.

---

12. `runAsync()`

Use `runAsync()` when the task doesn't return a value.

```
CompletableFuture<Void> future =
        CompletableFuture.runAsync(() -> {

            sendEmail();

        });
```

Comparison:

```
runAsync()
→ no result

supplyAsync()
→ returns result
```

---

13. Using a Custom Executor

Instead of:

```
CompletableFuture.supplyAsync(
        () -> getUser()
);
```

you can provide your own executor:

```
ExecutorService executor =
        Executors.newFixedThreadPool(4);

CompletableFuture<String> future =
        CompletableFuture.supplyAsync(
                () -> getUser(),
                executor
        );
```

This is often preferable when you want explicit control over the thread pool.

---

14. Synchronous vs Asynchronous Example

Suppose we have four services:

```
User Service           500ms
Order Service         1000ms
Payment Service        800ms
Recommendation Service 700ms
```

Synchronous

```
String user =
        userService.getUser(userId);

List<String> orders =
        orderService.getOrders(userId);

double payments =
        paymentService.getTotalPayments(userId);

List<String> recommendations =
        recommendationService
                .getRecommendations(userId);
```

Execution:

```
500 + 1000 + 800 + 700
= 3000ms
```

Asynchronous

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> userService.getUser(userId),
                executor
        );

CompletableFuture<List<String>> orderFuture =
        CompletableFuture.supplyAsync(
                () -> orderService.getOrders(userId),
                executor
        );

CompletableFuture<Double> paymentFuture =
        CompletableFuture.supplyAsync(
                () -> paymentService.getTotalPayments(userId),
                executor
        );

CompletableFuture<List<String>> recommendationFuture =
        CompletableFuture.supplyAsync(
                () -> recommendationService
                        .getRecommendations(userId),
                executor
        );
```

Now

Now:

```
User             ── 500ms
Orders           ────────── 1000ms
Payments         ──────── 800ms
Recommendations  ──────── 700ms
```

Approximately:

```
max(500, 1000, 800, 700)
≈ 1000ms
```

instead of:

```
3000ms
```

This is the key reason asynchronous concurrency is useful.

---

15. `thenApply()`

Use `thenApply()` to **transform a completed result**.

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> "Ramu"
        );

CompletableFuture<String> upperCaseFuture =
        userFuture.thenApply(
                String::toUpperCase
        );
```

Flow:

```
"Ramu"
   ↓
thenApply()
   ↓
"RAMU"
```

Think:

```
```

```
thenApply()
→ Transform result
```

---

16. `thenCompose()`

Use `thenCompose()` when the next asynchronous operation **depends on the previous result**.

Example:

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> getUser()
        );

CompletableFuture<List<String>> ordersFuture =
        userFuture.thenCompose(
                user ->
                        CompletableFuture.supplyAsync(
                                () -> getOrders(user)
                        )
        );
```

Flow:

```
getUser()
   ↓
User result
   ↓
thenCompose()
   ↓
getOrders(user)
   ↓
Orders
```

Mental model

```
A → B
```

where:

```
B depends on A
```

17. `thenCombine()`

Use `thenCombine()` when two independent futures need to be combined.

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> "Ramu"
        );

CompletableFuture<Integer> orderFuture =
        CompletableFuture.supplyAsync(
                () -> 5
        );

CompletableFuture<String> result =
        userFuture.thenCombine(
                orderFuture,
                (user, orders) ->
                        user + " has "
                                + orders
                                + " orders"
        );
```

Flow:

```
User Future ──────────┐
                      │
                      ├── thenCombine()
                      │
Orders Future ────────┘
                      ↓
               Combined Result
```

Mental model

```
A + B → C
```

18. `allOf()`

Use `allOf()` when you want to wait for multiple futures.

```
CompletableFuture.allOf(
        userFuture,
        orderFuture,
        paymentFuture,
        recommendationFuture
).join();
```

Conceptually:

```
User ──────────┐
Orders ────────┤
Payments ──────┤──→ allOf() → Continue
Recommendations ┘
```

Important:

`allOf()` returns:

```
CompletableFuture<Void>
```

So you retrieve the individual results separately:

```
String user = userFuture.join();

List<String> orders =
        orderFuture.join();
```

---

19. `join()` vs `get()`

Both can wait for a future.

`get()`

```
future.get();
```

throws checked exceptions.

`join()`

```
future.join();
```

throws unchecked `CompletionException`.

For `CompletableFuture`, `join()` is often convenient.

---

20. Exception Handling

Async operations can fail.

Example:

```
CompletableFuture<Double> paymentFuture =
        CompletableFuture.supplyAsync(
                () -> {

                    throw new RuntimeException(
                            "Payment service unavailable"
                    );
                },
                executor
        );
```

Without handling the exception, the future becomes exceptionally completed.

---

21. `exceptionally()`

Use `exceptionally()` to provide a fallback.

```
CompletableFuture<Double> paymentFuture =
        CompletableFuture
                .supplyAsync(
                        () -> paymentService
                                .getTotalPayments(userId),
                        executor
                )
                .exceptionally(
                        error -> 0.0
                );
```

Flow:

```
Payment Service
       ↓
    Failure
```

```
        ↓
exceptionally()
        ↓
   fallback 0.0
```

---

22. `handle()`

`handle()` receives both:

```
result
error
```

Example:

```
CompletableFuture<Double> paymentFuture =
        CompletableFuture
                .supplyAsync(
                        () -> paymentService
                                .getTotalPayments(userId),
                        executor
                )
                .handle(
                        (result, error) -> {

                            if (error != null) {
                                return 0.0;
                            }

                            return result;
                        }
                );
```

Comparison:

```
exceptionally()
→ mainly failure fallback

handle()
→ handle success + failure
```

---

23. Race Condition

A race condition happens when multiple threads access shared mutable data and the final result depends on execution timing.

Example:

```
class Counter {

    private int count = 0;

    public void increment() {
        count++;
    }
}
```

This looks simple:

```
count++;
```

but internally it's approximately:

```
read count
   ↓
increment
   ↓
write count
```

Two threads can interfere with each other.

---

24. `synchronized`

`synchronized` allows only one thread at a time to enter a synchronized method/block for the same monitor.

```
class Counter {

    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
```

```
}
```

Now:

```
Thread 1 → increment()
              ↓
           executes

Thread 2 → waits
```

---

25. AtomicInteger

For simple counters, `AtomicInteger` is often a cleaner solution.

```
AtomicInteger counter =
        new AtomicInteger(0);
```

Increment:

```
counter.incrementAndGet();
```

Read:

```
counter.get();
```

Example:

```
class Counter {

    private final AtomicInteger count =
            new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet();
    }

    public int getCount() {
        return count.get();
    }
}
```

Mental model:

Mental model:

```
int
 ↓
not automatically thread-safe

AtomicInteger
 ↓
thread-safe atomic operations
```

---

26. ConcurrentHashMap

Normal:

```
Map<Integer, String> map =
        new HashMap<>();
```

is not designed for concurrent modifications by multiple threads.

Use:

```
Map<Integer, String> map =
        new ConcurrentHashMap<>();
```

Example:

```
ConcurrentHashMap<Integer, String> users =
        new ConcurrentHashMap<>();

users.put(1, "Ramu");

String user =
        users.get(1);
```

Useful for:

- Concurrent caches
- Shared maps
- Multi-threaded applications

---

27. Thread Pool Lifecycle

Always remember:

```
ExecutorService executor =
        Executors.newFixedThreadPool(4);
```

At the end:

```
executor.shutdown();
```

Mental model:

```
Create Pool
    ↓
Submit Tasks
    ↓
Execute
    ↓
Shutdown
```

Not shutting down an executor can keep application threads alive unnecessarily.

---

28. Important Concurrency Patterns

Independent tasks

Use:

```
CompletableFuture.supplyAsync()
```

Example:

```
User
Orders
Payments
Recommendations
```

all run independently.

Transform result

Use:

```
thenApply()
```

```
A → transform → B
```

Dependent async operations

Use:

```
thenCompose()
```

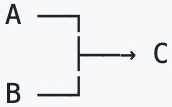
```
A → B
```

where B depends on A.

Combine independent results

Use:

```
thenCombine()
```



Wait for many tasks

Use:

```
allOf()
```

```
A ──┐
B ──┤
C ──┤ → allOf()
D ──┘
```

Handle failure

Use:

```
exceptionally()
```

or:

```
handle()
```

---

29. Complete Dashboard Example

This is the most important practical example from Level 6.

```
class Dashboard {

    private final ExecutorService executor =
            Executors.newFixedThreadPool(4);

    public DashboardResponse getDashboard(
            int userId
    ) {

        CompletableFuture<String> userFuture =
                CompletableFuture.supplyAsync(
                        () -> userService.getUser(userId),
                        executor
                );

        CompletableFuture<List<String>> ordersFuture =
                CompletableFuture.supplyAsync(
                        () -> orderService.getOrders(userId),
                        executor
                );

        CompletableFuture<Double> paymentFuture =
                CompletableFuture
                        .supplyAsync(
```

```
                                () -> paymentService
                                        .getTotalPayments(userId),
                                executor
                        )
                        .exceptionally(
                                error -> 0.0
                        );

        CompletableFuture<List<String>>
                recommendationFuture =
                CompletableFuture.supplyAsync(
                        () -> recommendationService
                                .getRecommendations(userId),
                        executor
                );

        CompletableFuture.allOf(
                userFuture,
                ordersFuture,
                paymentFuture,
                recommendationFuture
        ).join();

        return new DashboardResponse(
                userFuture.join(),
                ordersFuture.join(),
                paymentFuture.join(),
                recommendationFuture.join()
        );
    }

    public void shutdown() {
        executor.shutdown();
    }
}
```

Response:

```
record DashboardResponse(
        String user,
        List<String> orders,
        double payments,
        List<String> recommendations
) {
}
```

---

30. Synchronous → Asynchronous Conversion

This is the **most important practical pattern of Level 6**.

Before

```
String user =
        userService.getUser(userId);

List<String> orders =
        orderService.getOrders(userId);

double payments =
        paymentService.getTotalPayments(userId);
```

Execution:

```
User
 ↓
Orders
 ↓
Payments
```

After

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> userService.getUser(userId),
                executor
        );

CompletableFuture<List<String>> ordersFuture =
        CompletableFuture.supplyAsync(
                () -> orderService.getOrders(userId),
                executor
        );

CompletableFuture<Double> paymentsFuture =
        CompletableFuture.supplyAsync(
                () -> paymentService.getTotalPayments(userId),
                executor
        );
```

Execution:

```
        ┌── User
        │
```

```
Request├── Orders
       │
       └── Payments

      Concurrent
         ↓
      allOf()
         ↓
      Response
```

---

31. Important Rule — Don't Make Everything Async

Concurrency isn't automatically better.

Use asynchronous/concurrent execution when tasks are:

```
Independent
+
I/O bound
+
Can safely execute concurrently
```

Examples:

```
REST API calls
Database queries
External service calls
File I/O
Network operations
```

Don't blindly create asynchronous tasks for tiny CPU operations.

---

32. CPU-bound vs I/O-bound

I/O-bound

The program spends time waiting.

Examples:

```
Database
HTTP API
File
Network
```

Concurrency can provide significant benefits.

CPU-bound

The program spends time using the CPU.

Examples:

```
Image processing
Large calculations
Compression
Encryption
```

For CPU-bound work, simply increasing threads indefinitely does not improve performance.

---

33. Level 6 Cheat Sheet

```
Thread
  ↓
Basic execution unit

Runnable
  ↓
Task without result

Callable
  ↓
Task with result

ExecutorService
  ↓
Manages worker threads

Thread Pool
  ↓
Reuses threads

Future
  ↓
Represents future result
```

```
CompletableFuture
  ↓
Async + composable operations
```

CompletableFuture

```
runAsync()
    ↓
No result

supplyAsync()
    ↓
Returns result

thenApply()
    ↓
Transform result

thenCompose()
    ↓
Dependent async operation

thenCombine()
    ↓
Combine independent operations

allOf()
    ↓
Wait for multiple futures

exceptionally()
    ↓
Fallback on error

handle()
    ↓
Handle result + error

join()
    ↓
Wait/get result
```

Thread Safety

```
Race Condition
      ↓
synchronized
      ↓
AtomicInteger
```

```
AtomicInteger
      ↓
ConcurrentHashMap
```

🎯 Level 6 — What You Actually Need to Remember

Don't try to memorize every concurrency API.

For your backend development path, these are the **high-priority concepts**:

```
⭐⭐⭐⭐⭐

1. ExecutorService
2. Thread Pools
3. CompletableFuture
4. supplyAsync()
5. thenApply()
6. thenCompose()
7. thenCombine()
8. allOf()
9. Exception handling
10. synchronized
11. AtomicInteger
12. ConcurrentHashMap
```

The key mental model:

```
Independent work
      ↓
CompletableFuture
      ↓
Run concurrently
      ↓
Combine results
      ↓
Handle failures
      ↓
Return response
```

🧠 Level 6 Interview Questions

You should be able to answer these without looking at the notes:

You should be able to answer these without looking at the notes.

1. Why use ExecutorService instead of creating Threads manually?

Answer: It manages a reusable pool of threads and avoids the overhead and complexity of creating a new thread for every task.

2. `Runnable` vs `Callable`?

```
Runnable
→ no return value

Callable
→ returns a value
→ can throw checked exceptions
```

3. `Future` vs `CompletableFuture`?

```
Future
→ represents future result
→ mostly blocking with get()

CompletableFuture
→ async composition
→ chaining
→ combining
→ error handling
```

4. `thenApply()` vs `thenCompose()`?

```
thenApply()
→ transform result

thenCompose()
→ chain another async operation
```

5. `thenCombine()`?

Combines two independent asynchronous results.

6. Why use `allOf()`?

To wait for multiple independent `CompletableFuture`s to complete.

7. What is a race condition?

When multiple threads access shared mutable state and the result depends on the timing/order of execution.

8. `synchronized` vs `AtomicInteger`?

```
synchronized
→ protects critical sections

AtomicInteger
→ thread-safe atomic operations
→ ideal for simple counters
```

✅ Level 6 Completion Checklist

```
✅ Thread
✅ Runnable
✅ Thread.start()
✅ Thread.sleep()
✅ ExecutorService
✅ Fixed Thread Pool
✅ Callable
✅ Future
✅ CompletableFuture
✅ runAsync()
✅ supplyAsync()
✅ Custom Executor
✅ thenApply()
✅ thenCompose()
✅ thenCombine()
✅ allOf()
✅ join()
✅ Exception handling
✅ exceptionally()
✅ handle()
✅ Race Conditions
✅ synchronized
✅ AtomicInteger
✅ ConcurrentHashMap
✅ Executor shutdown
```